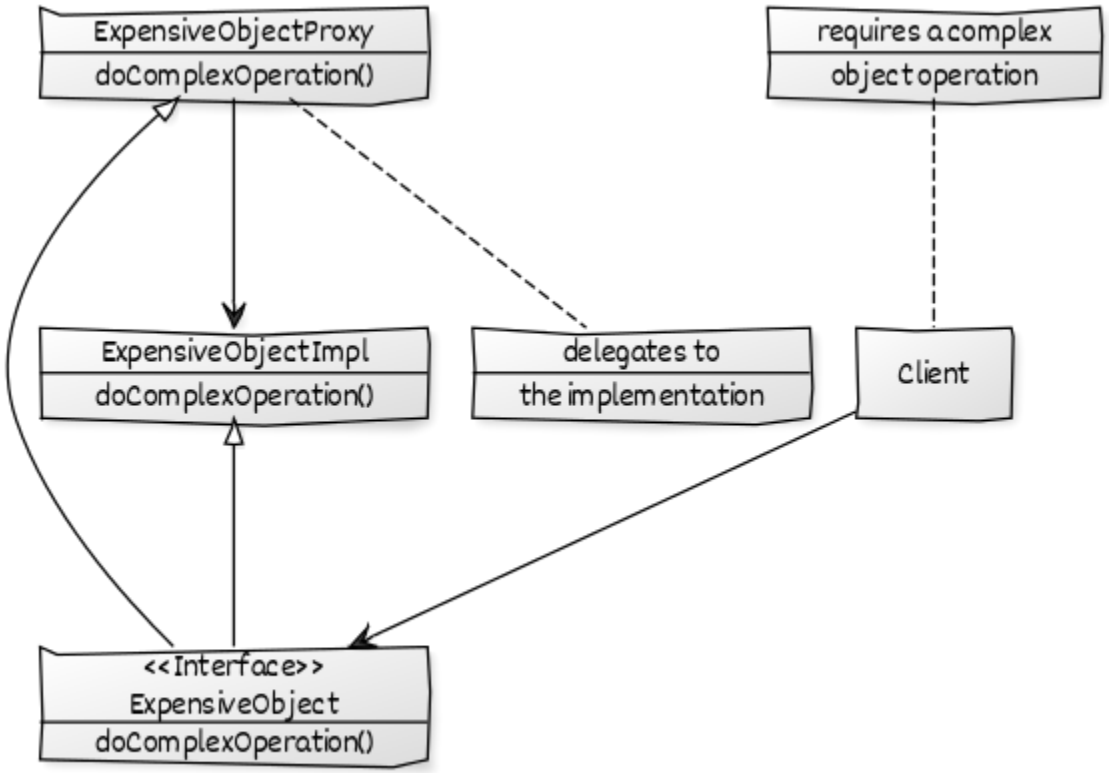
Proxy and Virtual Proxy / Lazy Initialization

- Creational pattern
- Shield the client from the actual implementation details of an object. Proxy can be derived in many flavors (security proxy, virtual proxy, …)
- Delay the creation of an object, the computation of a value, loading an object from DB, etc… until the first time it is needed.



CREATED WITH YUML

Proxy and Virtual Proxy / Lazy Initialization

```
public interface ExpensiveObject {
	void doComplexOperation();
}

public class ExpensiveObjectImpl implements ExpensiveObject {
	public ExpensiveObjectImpl() {
		heavyInitialConfiguration();
	}

	public void doComplexOperation() {
		LOG.info("processing complete.");
	}

	private void heavyInitialConfiguration() {
		LOG.info("Loading initial configuration...");
	}
}
```

```
public class ExpensiveObjectProxy implements ExpensiveObject {
	private static ExpensiveObject object;

	public void doComplexOperation() {
		if (object == null) {
			object = new ExpensiveObjectImpl();
		}
		object.process();
	}
}

public static void main(String[] args) {
	ExpensiveObject object = new ExpensiveObjectProxy();
	//Complex initialization is delayed until actual usage
	object. doComplexOperation();
}
```

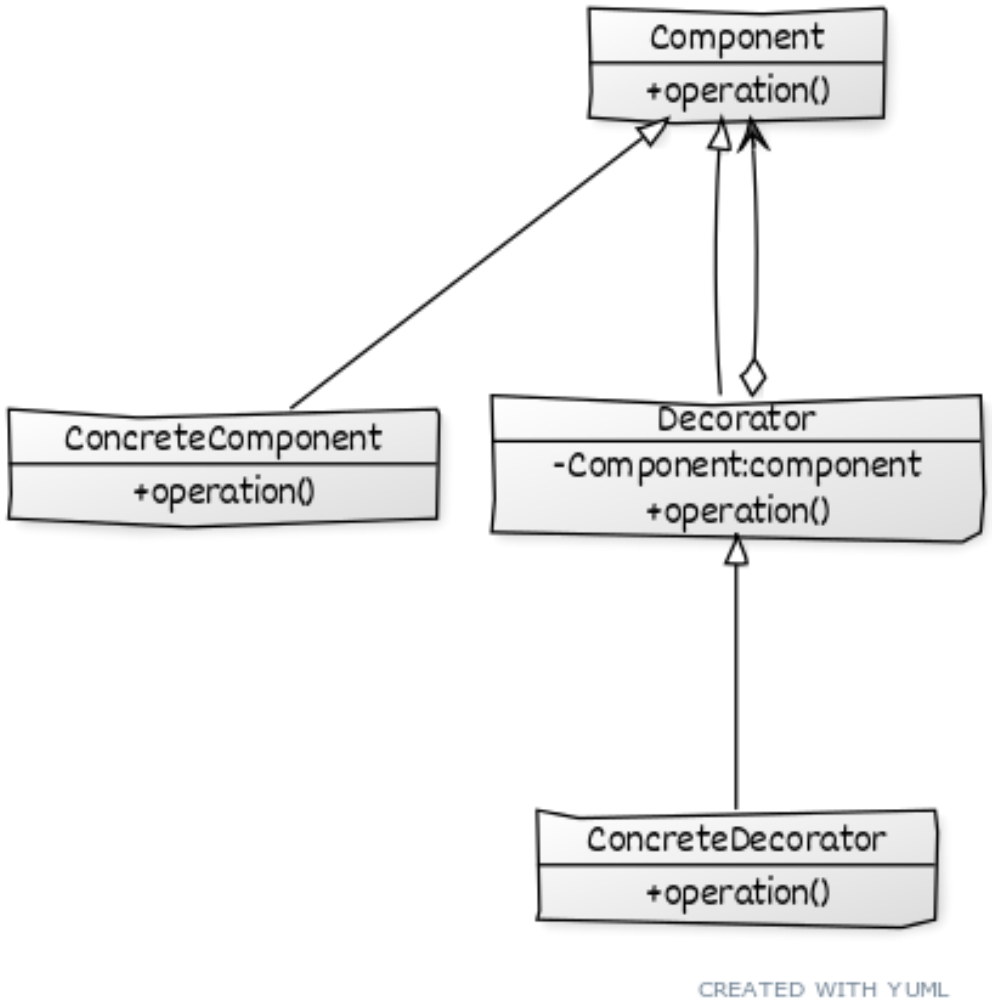
Proxy and Virtual Proxy / Lazy Initialization

```
public interface ExpensiveObject {
	VeryComplexObject getVeryComplexObject();
}

public class ExpensiveObjectImpl implements ExpensiveObject {

	public VeryComplexObject getVeryComplexObject() {
		LOG.info(“loading from db and building a complex tree.");
	}

}
```

```
public class ExpensiveObjectProxy implements ExpensiveObject {
	private static ExpensiveObject object =
				new ExpensiveObjectImpl();

	public VeryComplexObject getVeryComplexObject() {
		return object. getVeryComplexObject();
	}
}

public static void main(String[] args) {
	ExpensiveObject object = new ExpensiveObjectProxy();
	//Complex initialization is delayed until actual usage
	VeryComplexObject o = object. getVeryComplexObject();
}
```

Decorator

- Structural pattern
- Add responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.



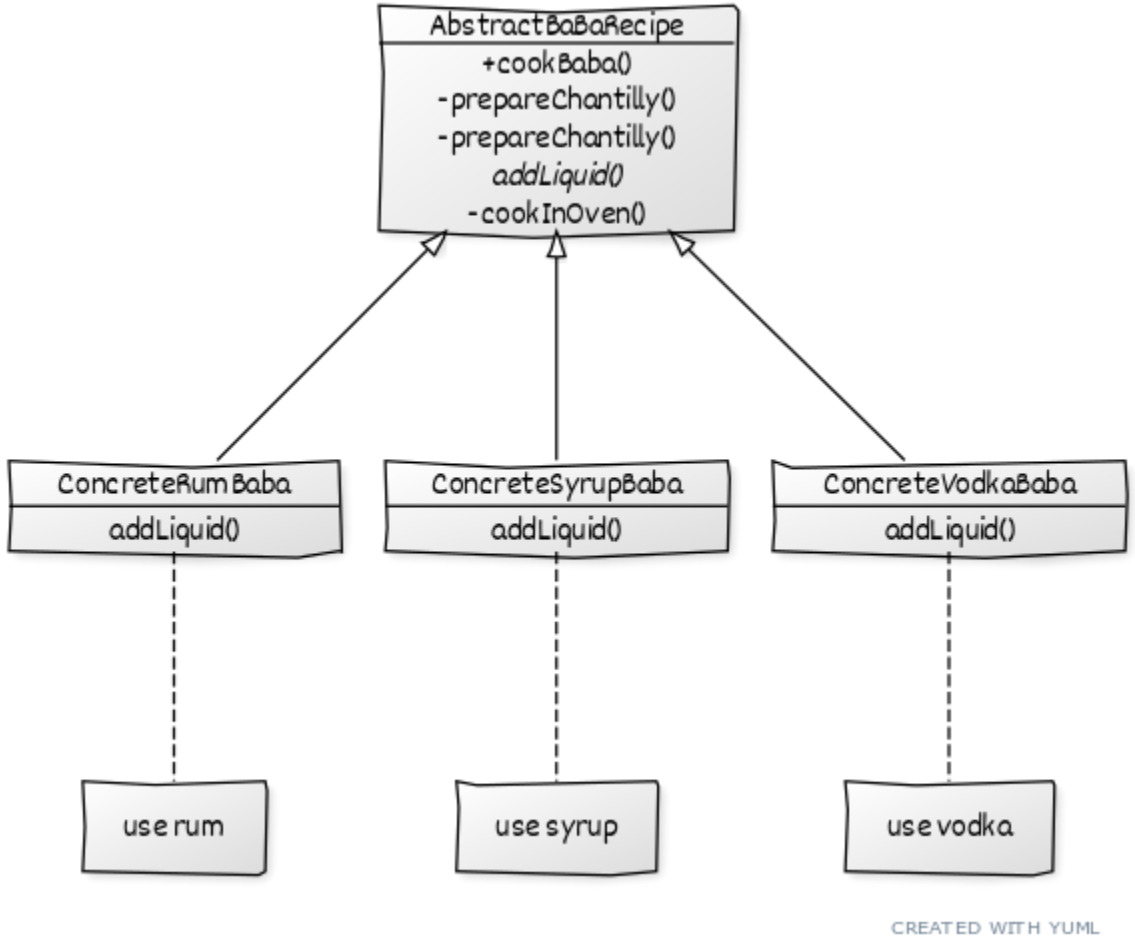
Decorator

```
public interface ChristmasTree { String decorate(); }

public class ChristmasTreeImpl implements ChristmasTree {
	public String decorate() { return "Christmas tree"; }
}

public abstract class TreeDecorator implements ChristmasTree {
	private ChristmasTree tree;
	public String decorate() { return tree.decorate(); }
}

public class BubbleLights extends TreeDecorator {
	public String decorate() {
		return super.decorate() + " with Bubbleights";
	}
}
```

```
public class Garland extends TreeDecorator {
	public String decorate() {
		return super.decorate() + " with Bubble Lights";
	}
}

public static void main(String[] args) {
	ChristmasTree tree = new BubbleLights(
		new Garland(new ChristmasTreeImpl())
	);
}
```

Template Method

- Behavioral Pattern
- Define the skeleton of an algorithm and define concrete implementations that vary



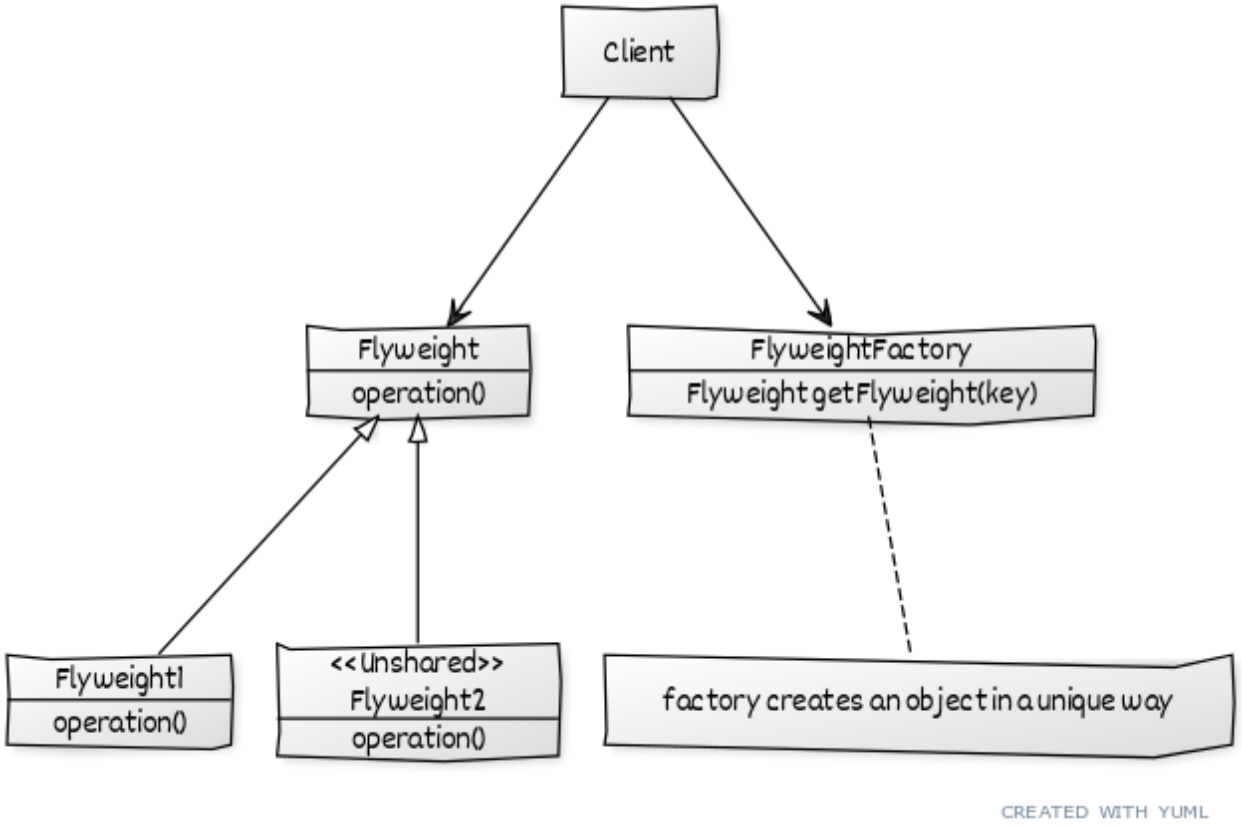
Template Method

```
static abstract class AbstractBaBaRecipe {
    public void cookBaba() {
        prepareDought();
        prepareChantilly();
        addLiquid();
        putInOven();
    }

    private void prepareDought() {
    }

    private void putInOven() {
    }

    protected abstract void addLiquid();

    private void prepareChantilly() {
    }
}
```

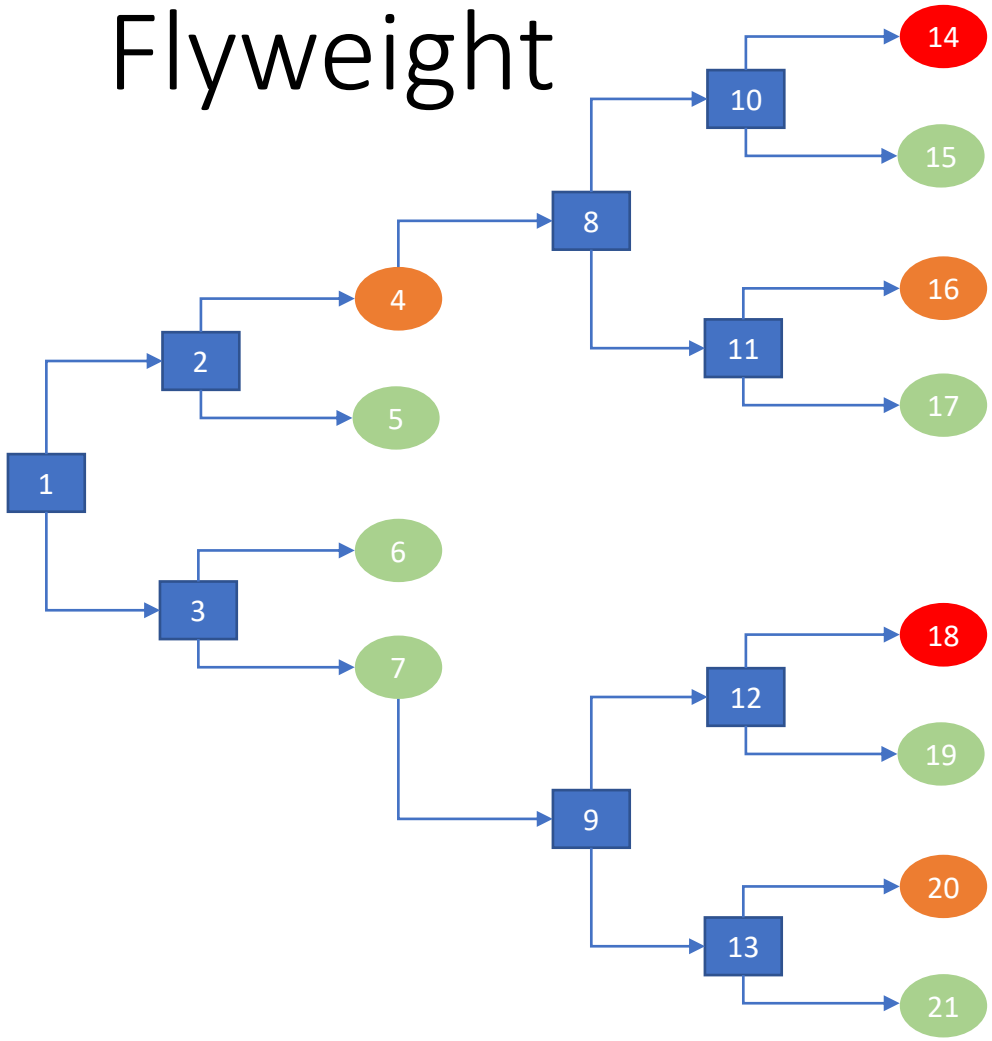
```
static class RumBaba extends AbstractBaBaRecipe {
    protected void addLiquid() {
    }
}

static class SyrupBaba extends AbstractBaBaRecipe {
    protected void addLiquid() {
    }
}

static class VodkaBaba extends AbstractBaBaRecipe {
    protected void addLiquid() {
    }
}
```

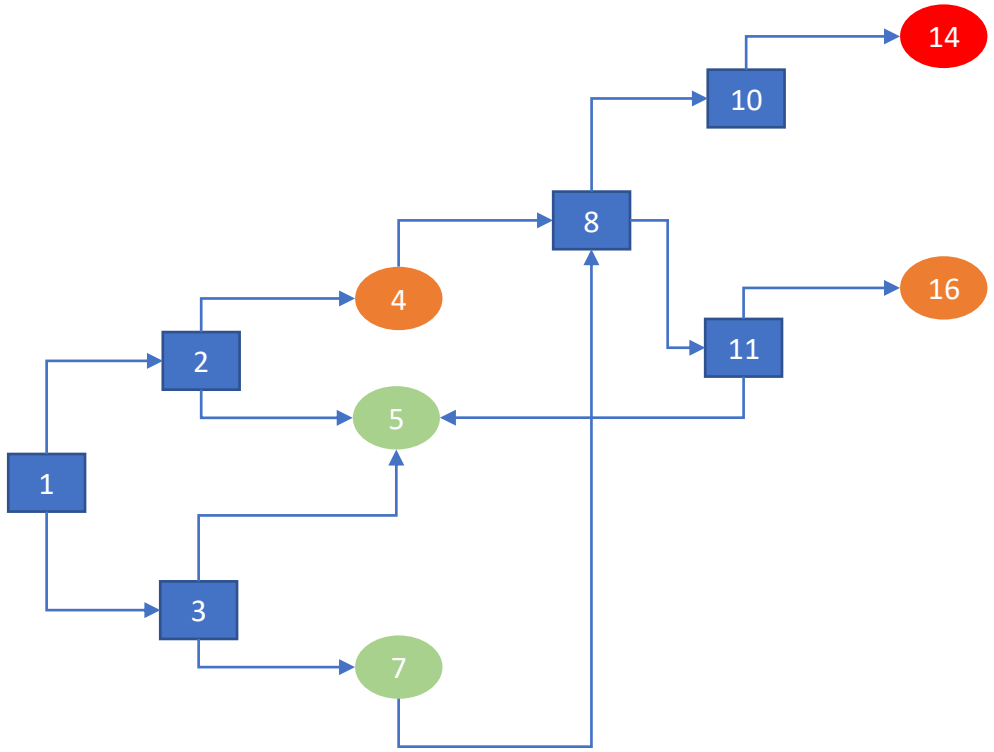
Flyweight

- **uses a large number of repeating** objects.
- objects are **expensive** to store.
- flyweight objects are immutable: any operation on the state must be performed by the factory.



Flyweight





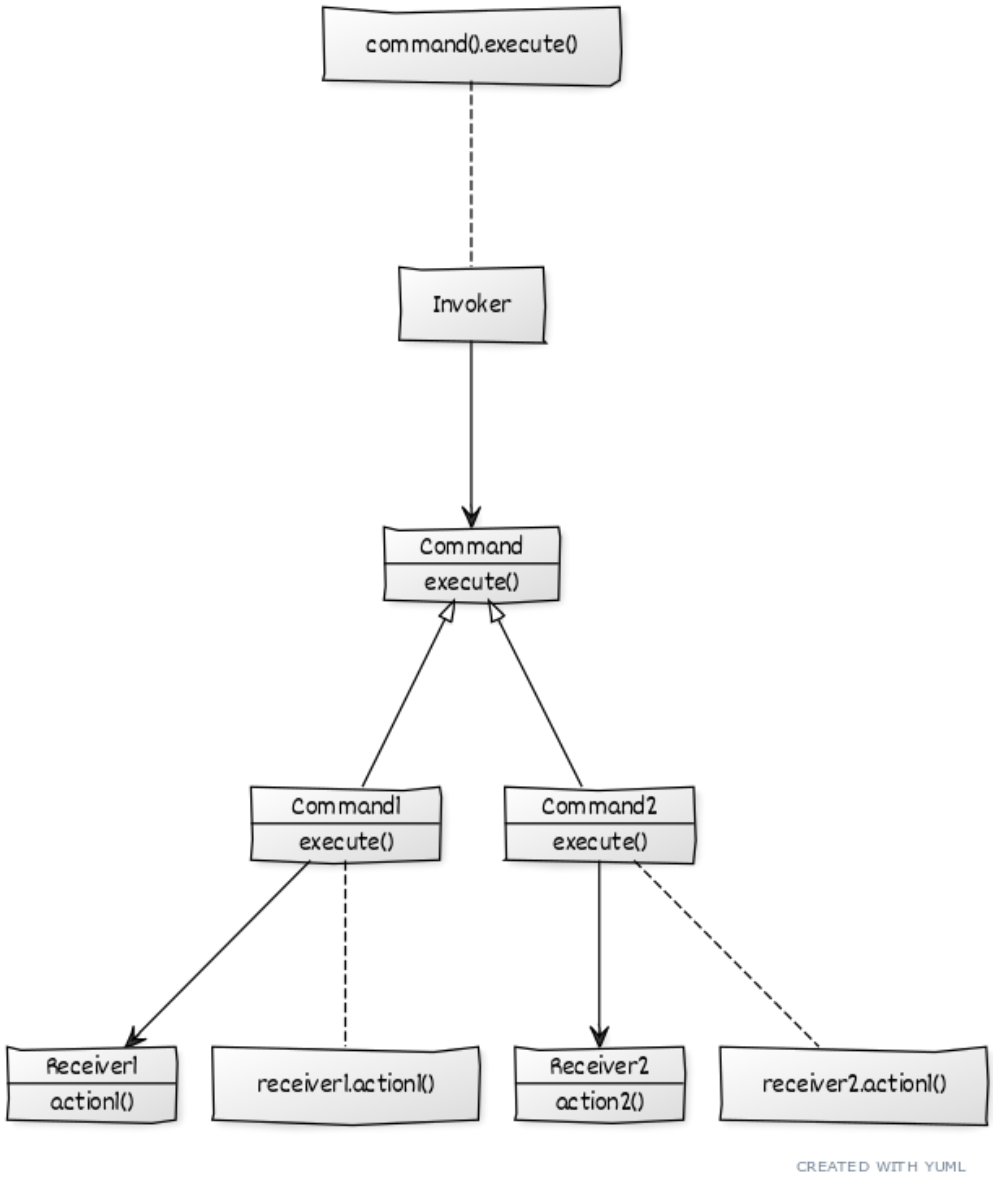
Flyweight

```java
static class Shape {

private String color;
private Shape[] children;
private static volatile Map<Shape, Shape> shapeCache = new WeakHashMap<>();

private Shape(String color, Shape[] children) {
     this.color = color;
     this.children = children;
}

private Shape(String color) {
     this.color = color;
     this.children = new Shape[0];
}

static synchronized Shape getInstance(String color, Shape... children) {
     List<Shape> uniqueChildren = Arrays.stream(children).filter(s ->
             s != null).map(s -> shapeCache.putIfAbsent(s,
             s)).collect(Collectors.toCollection(ArrayList::new));

     Shape s = new Shape(color, uniqueChildren.toArray(new Shape[0]));
     return shapeCache.computeIfAbsent(s, s1 -> s1);
}
```

```java
static int getShapeCount() {
     return shapeCache.size();
}
public String getColor() {
     return color;
}

public Shape[] getChildren() {
     return children;
}

public int hashCode() {
     return Arrays.hashCode(children) * 31 + Objects.hash(color);
}

public boolean equals(Object obj) {
     if (this == obj)
          return true;
     if (obj == null)
          return false;
     if (getClass() != obj.getClass())
          return false;
     Shape other = (Shape) obj;
     return Arrays.equals(children, other.children) && Objects.equals(color,
other.color);
}
```

Command

- Behavioral Pattern
- Queuing commands
- Undo/Redo operations
- Makes code extensible as we can add new commands without changing code.
- Reduces the coupling between the invoker and receiver of a command.



Command

```java
@FunctionalInterface
public interface Order {
      void execute();
}

public class Position {
      private String name;
      private int quantity;

      public Position(String name, int qty) {
            this.name = name;
            this.quantity = qty;
      }

      public void buy() {
            System.out.println("Position [ Name: " + name + ",
            Quantity: " + quantity + " ] bought");
      }

      public void sell() {
            System.out.println("Position [ Name: " + name + ",
            Quantity: " + quantity + " ] sold");
      }
      }
}
```

```java
   public class Broker {
   private List<Order> orderList = new ArrayList<Order>();

   public void takeOrder(Order order){
      orderList.add(order);
   }

   public void placeOrders(){
         orderList.forEach(Order::execute);
            orderList.clear();
   }
}


public static void main(String[] args) {
  CommandTest ct = new CommandTest();
  Position aaaPosition = ct.new Position("AAA", 10);
  Position bbbPosition = ct.new Position("BBB", 5);

      Broker broker = ct.new Broker();
      broker.takeOrder(aaaPosition::buy);
      broker.takeOrder(aaaPosition::buy);
      broker.takeOrder(bbbPosition::sell);
      broker.takeOrder(aaaPosition::sell);

      broker.placeOrders();
}
```

Compose patterns

Write code that combine singleton, template method, factory and command